

Student Meal Planner

Backend API Endpoints

An Implementation Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

Overview

The Grubs4Scrubs backend is an ASP.NET Core Web API running on localhost:5075. It uses a layered architecture with four projects: API (controllers), Business (services), DataAccess (repositories), and Domain (models). All data access uses raw ADO.NET. No Entity Framework.

This document lists every endpoint, what it does, what it expects, and what it returns. The base URL for all endpoints is `http://localhost:5075/api`.

Recipe Endpoints

All recipe endpoints go through `RecipeController`, which injects `IRecipeService`. The service handles validation, the repository handles SQL.

GET /api/recipes

Returns all recipes in the database.

Response: 200 OK with an array of Recipe objects.

```
[
  {
    "id": 1,
    "title": "Budget Pasta Carbonara",
    "description": "Classic carbonara with student-friendly ingredients",
    "prepTime": 10,
    "cookTime": 15,
    "servings": 2,
    "tag": "Dinner",
    "imageUrl": "/images/carbonara.jpg",
    "estimatedBudget": 3.50,
    "category": "Italian",
    "createdAt": "2026-04-01T12:00:00Z"
  }
]
```

GET /api/recipes/{id}

Returns a single recipe by its ID.

Response: 200 OK with a Recipe object, or 404 Not Found if the ID doesn't exist.

POST /api/recipes

Creates a new recipe.

Request body: a Recipe object (without Id, since that's auto-generated).

```
{
  "title": "Stir Fry Noodles",
  "description": "Quick noodles with whatever vegetables you have",
  "prepTime": 5,
  "cookTime": 10,
  "servings": 1,
  "tag": "Dinner",
  "imageUrl": null,
  "estimatedBudget": 2.80,
  "category": "Asian"
}
```

Response: 201 Created with the created Recipe (including the generated Id).

Validation (handled by RecipeService): - Title can't be empty. Returns 400 Bad Request if it is. - EstimatedBudget can't be negative. Returns 400 Bad Request if it is.

PUT /api/recipes/{id}

Updates an existing recipe.

Request body: a full Recipe object with all fields.

Response: 200 OK with the updated Recipe, or 404 Not Found if the ID doesn't match.

Validation: same as POST (title not empty, budget not negative), plus a check that the recipe exists before updating.

DELETE /api/recipes/{id}

Deletes a recipe by its ID.

Response: 204 No Content on success, or 404 Not Found if the ID doesn't exist.

The service checks that the recipe exists before calling the repository's delete method.

User Endpoints

User endpoints go through UserController, which injects IUserService. Same layered pattern as Recipe. The service validates, the repository runs the SQL. These endpoints are the scaffolding for the login flow. They cover the basic CRUD on the Users table. The actual authentication layer (password hashing, JWT issuing, Google OAuth) sits on top of this and is still being built.

The route uses [controller] so the base path is /api/user (singular). The User model has five fields that matter: Email, PasswordHash, UserName, GoogleId (nullable, for OAuth accounts), and CreatedAt.

GET /api/user

Returns every user in the database.

Response: 200 OK with an array of User objects.

```
[
  {
    "id": 1,
    "email": "boss@grubs4scrubs.local",
    "passwordHash": "$2a$11$PLACEHOLDERHASHBOSS",
    "userName": "boss",
    "googleId": null,
    "createdAt": "2026-04-16T09:00:00Z"
  }
]
```

One heads-up: this endpoint returns the PasswordHash field too, which isn't something a real production API should ever do. It's fine for development while the frontend login form is being wired up, but the response shape will change (to a DTO without the hash) before any of this ships.

GET /api/user/{id}

Returns a single user by their ID.

Response: 200 OK with a User object, or 404 Not Found if the ID isn't there.

POST /api/user

Creates a new user. Request body is a User object without an Id.

```
{
  "email": "newstudent@grubs4scrubs.local",
  "passwordHash": "$2a$11$somethingbcrypt",
  "userName": "newstudent",
  "googleId": null
}
```

Response: 201 Created.

Validation (handled by UserService): - Email can't be empty. 400 Bad Request if it is. - UserName can't be empty. 400 Bad Request. - PasswordHash can't be empty. 400 Bad Request.

Right now the endpoint trusts the client to send an already-hashed password, which is only OK because there's no real client yet. Once the register form exists, the hashing moves into the service layer, the controller stops accepting a raw PasswordHash, and this endpoint gets replaced (or wrapped) by `/api/auth/register`. For now it's a direct CRUD insert.

PUT /api/user/{id}

Updates an existing user. Request body is a full User object.

Response: 204 No Content on success, 404 Not Found if the ID doesn't exist.

The service checks the user exists before hitting the repository. Same pattern as Recipe.

DELETE /api/user/{id}

Deletes a user by ID.

Response: 204 No Content on success, 404 Not Found if the ID doesn't exist.

No cascade logic yet. Once meal plans and shopping lists are tied to users, this endpoint will need to either block deletion or clean up the related rows.

How the Layers Connect

A request hits RecipeController, which calls IRecipeService. The service validates the input and calls IRecipeRepository. The repository opens a SqlConnection, runs a parameterized SqlCommand, and returns mapped objects. The controller never touches the database directly.

```
HTTP Request
→ RecipeController (API layer)
```

```
→ IRecipeService / RecipeService (Business layer)
→ IRecipeRepository / RecipeRepository (DataAccess layer)
→ SQL Server via SqlConnection + SqlCommand
```

All of this is wired through dependency injection in Program.cs:

```
builder.Services.AddScoped<IRecipeRepository, RecipeRepository>();
builder.Services.AddScoped<IRecipeService, RecipeService>();
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IShoppingItemRepository, ShoppingItemRepository>();
builder.Services.AddScoped<IShoppingItemService, ShoppingItemService>();
```

The User stack follows the exact same shape as Recipe. UserController → IUserService → IUserRepository → SqlConnection. If you understand one flow, you understand the other.

Database Connection

Connection string lives in appsettings.json under "DefaultConnection". Points to localhost SQL Server, database name Grubs4Scrubs. Repositories read it through IConfiguration injected in their constructors.

CORS

The API has CORS configured to accept requests from `http://localhost:5173` (the Vite dev server). Without this, the React frontend can't call the API because browsers block cross-origin requests by default.

Auth Endpoints

Auth endpoints live in AuthController. This controller doesn't follow the standard service/repository pattern. It handles its own logic directly: BCrypt hashing, credential verification, and JWT token generation. It injects IUserRepository (to check for existing emails and create users) and IConfiguration (to read JWT settings from appsettings.json).

POST /api/auth/register

Registers a new user. Hashes the password with BCrypt before storing it.

Request body:

```
{
  "userName": "boss",
  "email": "boss@grubs4scrubs.local",
  "password": "mypassword123"
}
```

Response: 200 OK on success. 400 Bad Request if the email is already in use.

The password field in the DTO is plain text. The controller hashes it with `BCrypt.Net.BCrypt.HashPassword()` before passing it to the repository. The client never sends or sees a hash.

POST /api/auth/login

Verifies credentials and returns a JWT token.

Request body:

```
{
  "email": "boss@grubs4scrubs.local",
  "password": "mypassword123"
}
```

Response on success: 200 OK with a token object.

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response on failure: 401 Unauthorized with "Invalid email or password". The error message is intentionally the same whether the email doesn't exist or the password is wrong. This prevents email enumeration (an attacker can't figure out which emails are registered by checking error messages).

The token contains three claims: `NamIdentifier` (user ID), `Email`, and `Name` (username). It's signed with HMAC-SHA256 using a symmetric key from `appsettings.json` and expires after 24 hours.

Shopping Item Endpoints

Shopping item endpoints go through `ShoppingItemController`, which injects `IShoppingItemService`. Same layered pattern as `Recipe` and `User`.

GET /api/shoppingitem

Returns all shopping items.

Response: 200 OK with an array of ShoppingItem objects.

```
[
  {
    "id": 1,
    "name": "Pasta",
    "quantity": "500g",
    "price": 1.29,
    "isChecked": false,
    "userId": null
  }
]
```

GET /api/shoppingitem/{id}

Returns a single shopping item by ID.

Response: 200 OK or 404 Not Found.

POST /api/shoppingitem

Creates a new shopping item.

Request body:

```
{
  "name": "Rice",
  "quantity": "1kg",
  "price": 2.49,
  "isChecked": false,
  "userId": null
}
```

Response: 201 Created with the created item.

PUT /api/shoppingitem/{id}

Updates an existing shopping item. Used by the inline edit feature on the frontend.

Request body: full ShoppingItem object with updated values.

Response: 200 OK on success, 404 Not Found if the ID doesn't exist.

DELETE /api/shoppingitem/{id}

Deletes a shopping item by ID.

Response: 204 No Content on success, 404 Not Found if the ID doesn't exist.

Planned Endpoints (Not Yet Built)

Google OAuth: - POST /api/auth/google (Google OAuth login, populates GoogleId on the user)

Meal Plan endpoints: - GET /api/mealplans/{userId} (get a user's meal plan) - POST /api/mealplans (save or update a meal plan)

Error Handling

Right now error handling is basic. The service layer throws exceptions for validation failures, and the controller catches them and returns appropriate HTTP status codes. 400 for bad input, 404 for not found, 500 for unexpected errors. This will need to get more specific as more endpoints are added, probably with a custom exception class or middleware.

JWT Authentication Middleware

Program.cs configures JWT Bearer authentication. The middleware validates incoming tokens against the key, issuer, and audience defined in appsettings.json. The pipeline order matters: `UseAuthentication()` runs before `UseAuthorization()`. Right now no endpoints are marked with `[Authorize]` yet, but the middleware is in place and ready. Once the frontend sends the JWT in the Authorization header, protected endpoints will just need the attribute added.

Frontend Integration

The React frontend calls these endpoints through Axios. The API service file lives at `src/services/recipeservice.js` with a base URL of `http://localhost:5075/api`. Recipe CRUD (including edit via PUT and delete) and shopping item CRUD are fully wired. The shopping list page uses inline editing with PUT requests. The recipe view page uses a modal for editing.

LoginPage and SignUpPage components exist on the frontend but aren't connected to the auth endpoints yet. That's planned for the next submission. The backend auth system (register, login, JWT generation) is fully functional and testable through Swagger or Postman.